

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No: 8

Student Name: M. Vanisree

Roll No.: A24126552125

Date: 06-10-2026

Title

Implementation of HTTP Methods using JSON Server

Aim

To implement and demonstrate the GET, POST, PUT and DELETE HTTP methods using JSON Server for managing data in a RESTful web application.

Objectives

1. To understand the concept of a REST API and JSON Server.
 2. To create a simple JSON-based REST API.
 3. To perform GET, POST, PUT and DELETE operations.
 4. To understand how client applications communicate with a server using HTTP methods.
 5. To test API endpoints using a suitable tool such as Postman or a web browser.
-

Theory

JSON Server is a lightweight tool that allows developers to create a REST API from a JSON file without developing a complete backend server. It uses HTTP methods to perform operations on resources.

HTTP Method	Purpose	Example
GET	Retrieve data	Get all students
POST	Add new data	Add a student
PUT	Update existing data	Update student details
DELETE	Remove data	Delete a student

For this experiment, a db.json file is used as the database. When JSON Server is started, it automatically provides REST endpoints such as GET /students, GET /students/1, POST /students, PUT /students/1 and DELETE /students/1. Thus, JSON Server helps students understand RESTful API operations before working with a full backend framework.

Concepts Used

Concept	Purpose
JSON Server	Creates a REST API from a JSON file.
REST API	Provides endpoints through which client applications communicate with resources.
GET	Retrieves student records.
POST	Adds a new student record.
PUT	Updates an existing student record.
DELETE	Removes a student record.
fetch()	Sends HTTP requests from the browser to the JSON Server API.
async / await	Handles asynchronous API operations in a readable way.
JSON.stringify()	Converts a JavaScript object into JSON text for a request body.
response.json()	Converts the JSON response into a JavaScript object or array.

Algorithm / Procedure

Step 1: Install Node.js and npm on the system.

Step 2: Create a new project folder named JSONServerDemo.

Step 3: Open the terminal in the project folder and initialize the project using npm init -y.

Step 4: Install JSON Server using npm install json-server.

Step 5: Create a file named db.json.

Step 6: Add sample student data to db.json.

Step 7: Start JSON Server using npx json-server --watch db.json --port 3000.

Step 8: Test the GET operation to retrieve all students.

Step 9: Test the POST operation to add a new student.

Step 10: Test the PUT operation to modify an existing student's details.

Step 11: Test the DELETE operation to remove a student.

Step 12: Verify the changes in db.json and record the outputs.

Program / Code

1. db.json

```
{
  "students": [
    {
      "id": "1",
```

```
"name": "Ravi",
"branch": "CSE",
"marks": 85
},
{
  "id": "2",
  "name": "Priya",
  "branch": "AIML",
  "marks": 91
},
{
  "id": "3",
  "name": "Kiran",
  "branch": "CSE",
  "marks": 78
}
]
}
```

2. index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Student Management System</title>
<style>
body {
font-family: Arial, sans-serif;
margin: 40px;
background-color: #f4f4f4;
}

h1 {
text-align: center;
}

.form-container {
background: white;
padding: 20px;
margin-bottom: 25px;
border-radius: 8px;
}

input {
padding: 10px;
margin: 5px;
}
```

```

button {
padding: 9px 14px;
margin: 5px;
cursor: pointer;
}

table {
width: 100%;
background: white;
border-collapse: collapse;
}

th,
td {
padding: 12px;
border: 1px solid #ccc;
text-align: center;
}

th {
background-color: #333;
color: white;
}
</style>
</head>

<body>
<h1>Student Management System</h1>

<div class="form-container">
<h2>Add Student</h2>
<input type="text" id="name" placeholder="Student Name">
<input type="text" id="branch" placeholder="Branch">
<input type="number" id="marks" placeholder="Marks">
<button onclick="addStudent()">Add Student</button>
<button onclick="getStudents()">Refresh</button>
</div>

<div class="form-container">
<h2>Student Records</h2>
<table>
<thead>
<tr>
<th>ID</th>
<th>Name</th>
<th>Branch</th>
<th>Marks</th>

```

```

<th>Actions</th>
</tr>
</thead>
<tbody id="studentTable">
</tbody>
</table>
</div>

<script src="script.js"></script>
</body>
</html>

```

3. script.js

```

const API_URL = "http://localhost:3000/students";

// GET: Display all students
async function getStudents() {
  try {
    const response = await fetch(API_URL);
    const students = await response.json();

    const table = document.getElementById("studentTable");
    table.innerHTML = "";

    students.forEach(student => {
      const row = document.createElement("tr");

      row.innerHTML = `
        <td>${student.id}</td>
        <td>${student.name}</td>
        <td>${student.branch}</td>
        <td>${student.marks}</td>
        <td>
          <button onclick="updateStudent('${student.id}')">
            Update
          </button>
          <button onclick="deleteStudent('${student.id}')">
            Delete
          </button>
        </td>
      `;

      table.appendChild(row);
    });
  } catch (error) {
    console.error("Error:", error);
    alert("Unable to fetch student records.");
  }
}

```

```

}

// POST: Add a new student
async function addStudent() {
  const name = document.getElementById("name").value;
  const branch = document.getElementById("branch").value;
  const marks = document.getElementById("marks").value;

  if (name === "" || branch === "" || marks === "") {
    alert("Please enter all student details.");
    return;
  }

  const student = {
    name: name,
    branch: branch,
    marks: Number(marks)
  };

  try {
    const response = await fetch(API_URL, {
      method: "POST",
      headers: {
        "Content-Type": "application/json"
      },
      body: JSON.stringify(student)
    });

    if (response.ok) {
      alert("Student added successfully.");

      document.getElementById("name").value = "";
      document.getElementById("branch").value = "";
      document.getElementById("marks").value = "";

      getStudents();
    }
  } catch (error) {
    console.error("Error:", error);
    alert("Unable to add student.");
  }
}

// PUT: Update student
async function updateStudent(id) {
  try {
    // Get current student details
    const response = await fetch(`${API_URL}/${id}`);
  }

```

```

const student = await response.json();

// Ask for updated values
const name = prompt("Enter student name:", student.name);
if (name === null) {
    return;
}

const branch = prompt("Enter branch:", student.branch);
if (branch === null) {
    return;
}

const marks = prompt("Enter marks:", student.marks);
if (marks === null) {
    return;
}

const updatedStudent = {
    name: name,
    branch: branch,
    marks: Number(marks)
};

// Update using PUT
const updateResponse = await fetch(`${API_URL}/${id}`, {
    method: "PUT",
    headers: {
        "Content-Type": "application/json"
    },
    body: JSON.stringify(updatedStudent)
});

if (updateResponse.ok) {
    alert("Student updated successfully.");
    getStudents();
}
} catch (error) {
    console.error("Error:", error);
    alert("Unable to update student.");
}
}

// DELETE: Delete student
async function deleteStudent(id) {
    const confirmDelete = confirm(
        "Are you sure you want to delete this student?"
    );
};

```

```
if (!confirmDelete) {
  return;
}

try {
  const response = await fetch(`${API_URL}/${id}`, {
    method: "DELETE"
  });

  if (response.ok) {
    alert("Student deleted successfully.");
    getStudents();
  }
} catch (error) {
  console.error("Error:", error);
  alert("Unable to delete student.");
}

// Load students when page opens
getStudents();
```

HTTP Operations

A. GET Method

Retrieve all students

Method: GET

B. POST Method

Add a new student

Method: POST

```
{
  "name": "Anil",
  "branch": "CSE",
  "marks": 88
}
```

C. PUT Method

Update student details. The details of student with ID 1 will be updated.

Method: PUT

D. DELETE Method

Delete a student.

Method: DELETE

http://localhost:3000/students/3 - The student with ID 3 will be removed.

Test Cases and Execution Data

The following test cases are based on the execution data provided in the source experiment.

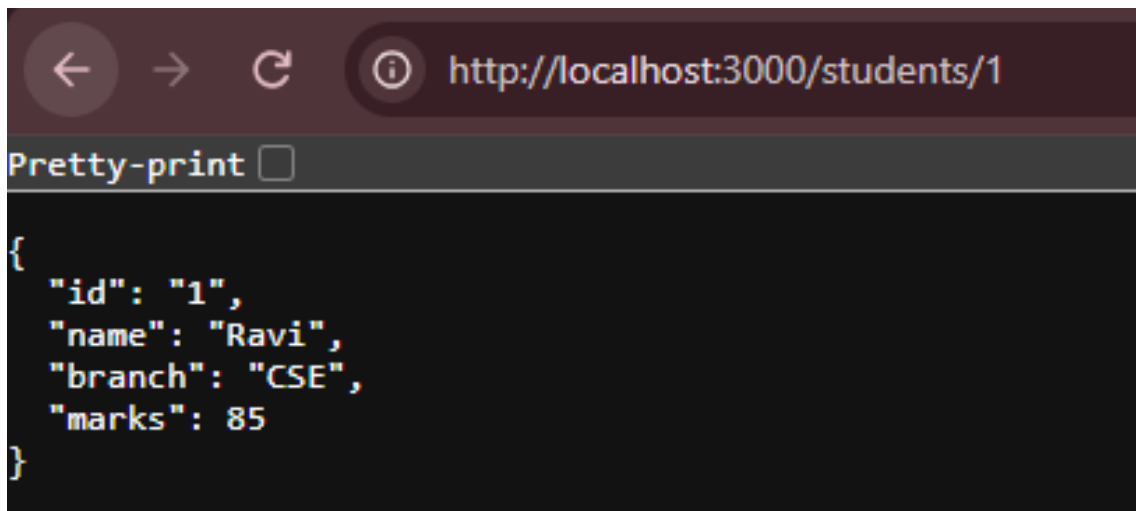
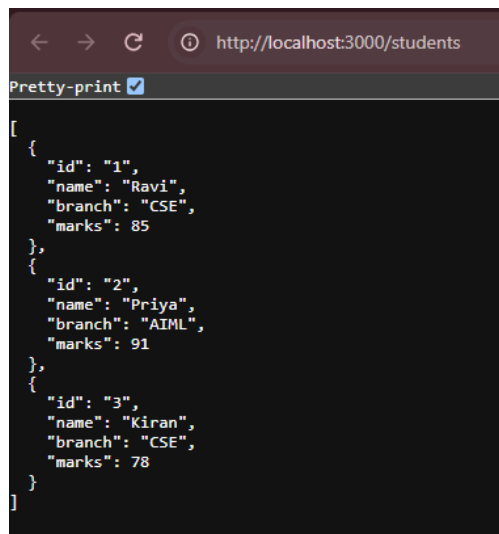
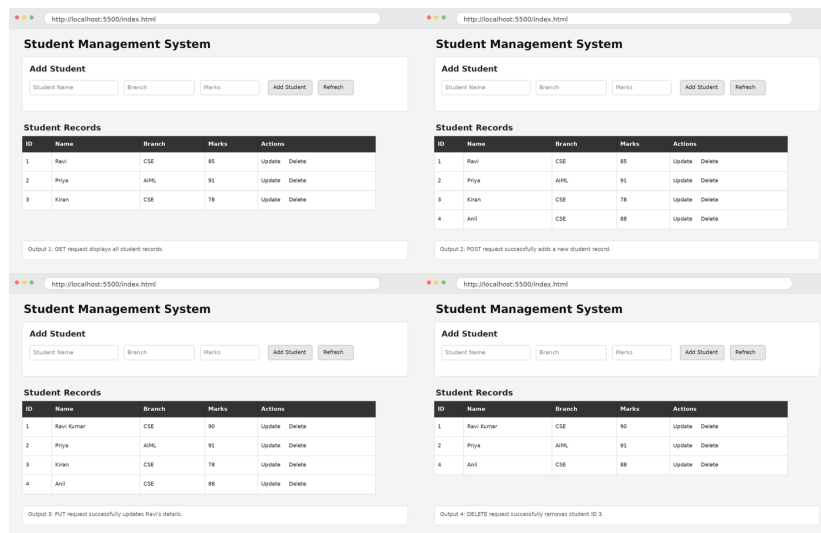
Test Case	HTTP Method	URL	Input / Operation	Expected Result
TC01	GET	/students	Retrieve all students	All student records are displayed
TC02	GET	/students/1	Retrieve student ID 1	Student ID 1 details are displayed
TC03	POST	/students	Add Anil, CSE, 88	New student record is created
TC04	PUT	/students/1	Change Ravi's marks to 90	Student ID 1 is updated
TC05	DELETE	/students/3	Delete student ID 3	Student ID 3 is deleted

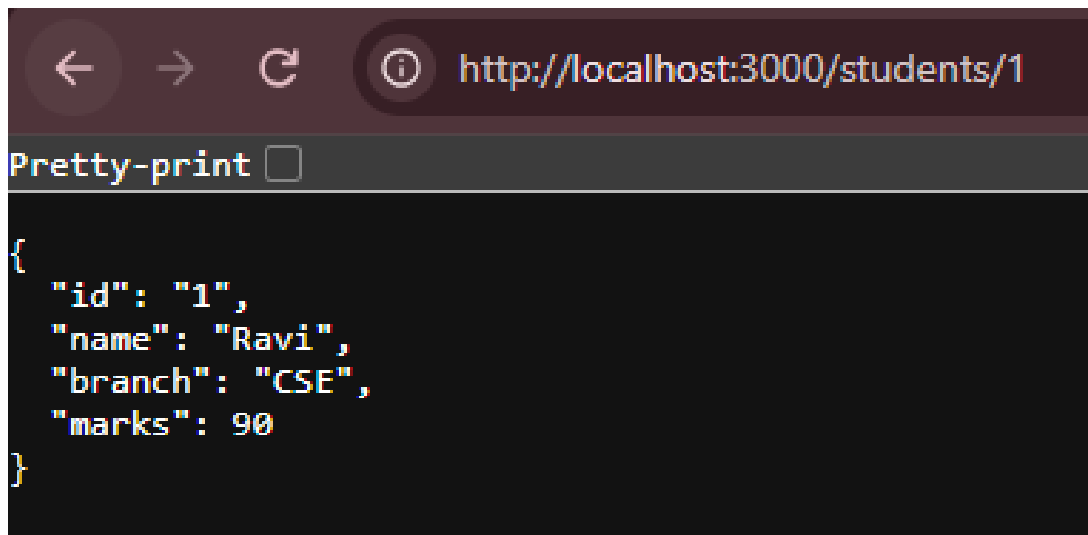
Execution Data

Operation	Input	Actual Output	Status
GET all students	/students	3 student records displayed	Pass
GET student	/students/1	Ravi's details displayed	Pass
POST student	Anil, CSE, 88	New record added	Pass
PUT student	ID 1, marks = 90	Record updated	Pass
DELETE student	ID 3	Record deleted	Pass

Output

The executed application displays the Student Management System interface and demonstrates student records being retrieved and manipulated through the JSON Server REST endpoints.





Result

Thus, the GET, POST, PUT and DELETE HTTP methods were successfully implemented and tested using JSON Server for managing student records.
